Reporte Técnico de Actividades Práctico-Experimentales Nro. 005

1. Datos Generales

Nombre del estudiante(s)	Israel Rodríguez Douglas Carreño Jossibel Perez Jorge Luzuriaga
Asignatura	Sistemas distribuidos
Ciclo	6to
Unidad	1
Resultado de aprendizaje de la unidad	R2. Escribe un programa que realiza cualquier requerimiento de clasificación y conversión en unidades de mensajes, como paquetes, para comunicar información interesante entre dos hosts, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	005
Título de la Práctica	Mensajería Distribuida con RabbitMQ y MQTT
Nombre del Docente	José O. Guamán Q.
Fecha	20 de Octubre de 2026 23 de Octubre de 2026
Horario	Martes 10h30 – 11h30 VIernes 07h30 – 09h30
Lugar	Aula
Tiempo planificado en el Sílabo	3 horas

2. Objetivo(s) de la Práctica

Implementar la capa IoT de un sistema de monitoreo de flota logística utilizando el protocolo MQTT: configurar un broker Mosquitto en Docker, simular sensores que publican telemetría en topics jerárquicos con paho-mqtt, implementar un suscriptor que almacene los datos en SQLite y genere alertas por umbral, y construir un servicio puente que publique las alertas en colas de RabbitMQ.

- Comprender los fundamentos del protocolo MQTT y su aplicación en entornos IoT

- Configurar e instalar un broker Mosquitto para mensajeria publicador/suscriptor
- Implementar publicadores y suscriptores MQTT en Python usando paho-mqtt
- Disenar topic hierarchies para organizar la telemetria de una flota vehicular
- Entender el protocolo AMQP 0-9-1 y el modelo de intercambio de RabbitMQ
- Configurar exchanges, colas y bindings en RabbitMQ.
- Implementar microservicios en Spring Boot que consuman mensajes de RabbitMQ
- Integrar MQTT y RabbitMQ mediante un servicio puente en Python
- Aplicar patrones de mensajería: pub/sub, routing y competing consumers.

3. Materiales y Reactivos:

Hardware
- Computador

Software
- Python
- paho-mqtt
- pika
- Java JDK 17
- Spring Boot
- Docker
- Docker Compose
- Mosquitto
- RabbitMQ
- SQLite3
- IDE
- Postman
- Navegador web

4. Procedimiento / Metodología

La práctica se desarrolló en dos sesiones (martes: entorno y simulador; viernes: persistencia, alertas y RabbitMQ) sobre Debian 12, con Docker y Python 3.11. La arquitectura implementada sigue el flujo: **Simulador de sensores → Mosquitto (MQTT) → Suscriptor + SQLite** y, en paralelo, **Mosquitto → Puente → RabbitMQ (AMQP)**.

Actividad 1 – Configuración del entorno

- Se creó el proyecto con `docker-compose.yml`, `mosquitto/config/mosquitto.conf` y `requirements.txt` (paho-mqtt ≥ 2.0 y pika ≥ 1.3).
- Como Debian aplica la restricción PEP 668, las librerías se instalaron en un entorno virtual: `python3 -m venv venv`, `source venv/bin/activate` y `pip install -r requirements.txt`.

- El archivo `docker-compose.yml` levanta dos servicios: Mosquitto (`eclipse-mosquitto:2.0`, puertos 1883 y 9001) y RabbitMQ (`rabbitmq:3-management`, puertos 5672 y 15672).
- En `mosquitto.conf` se habilitó el listener 1883 (MQTT), el listener 9001 (WebSockets), el acceso anónimo para el laboratorio y la persistencia en `/mosquitto/data/`.
- Incidencias resueltas: (1) el puerto 1883 estaba ocupado por un servicio Mosquitto instalado en el sistema, que se detuvo con `systemctl stop mosquitto`; (2) RabbitMQ se reiniciaba en bucle con el error "Too short cookie string", porque el disco estaba lleno al generar la cookie de Erlang. Se liberó espacio (`docker system prune`, `docker builder prune`) y se recreó el contenedor.
- Se verificó el broker publicando "Hola MQTT" en el topic `test/topic` y recibiéndolo con `mosquitto_sub`.

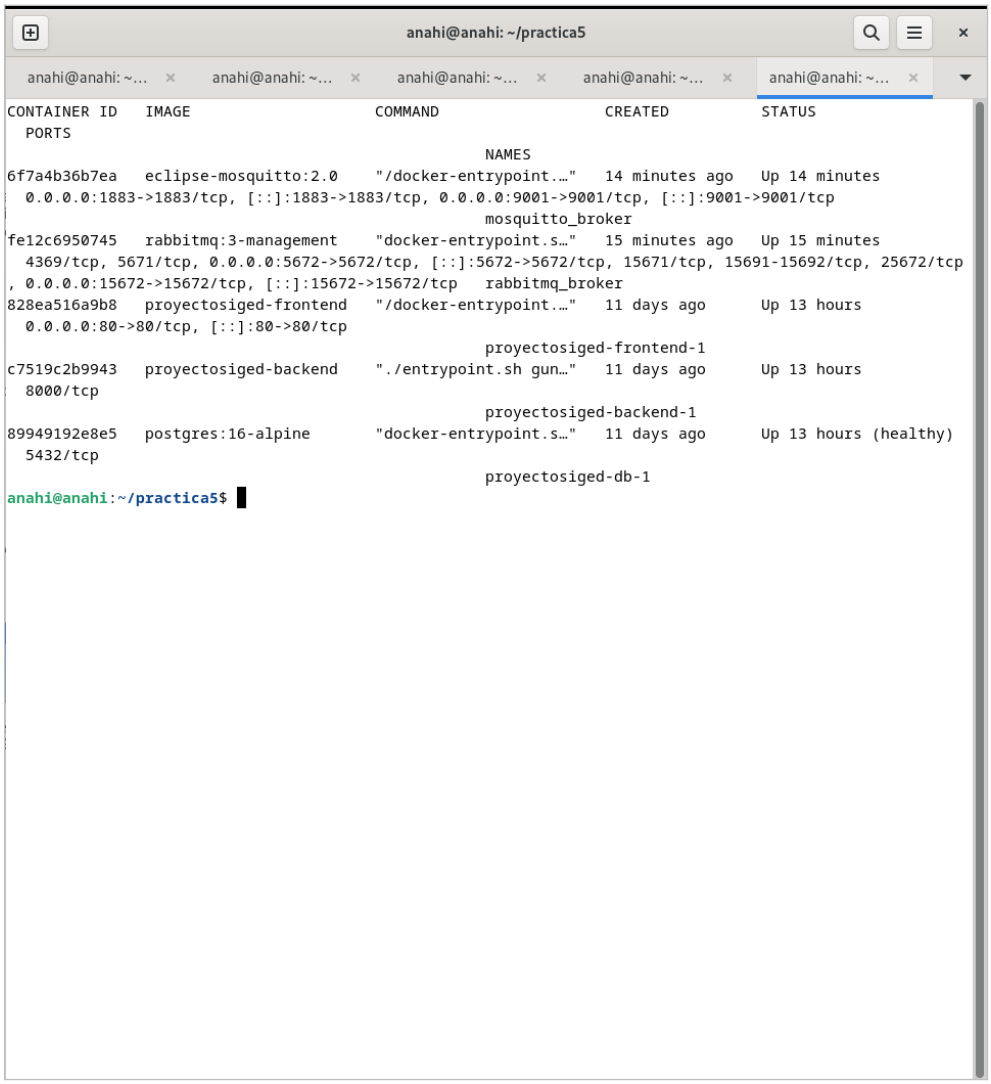


Figura 1. Contenedores mosquitto_broker y rabbitmq_broker en estado Up con sus puertos publicados.

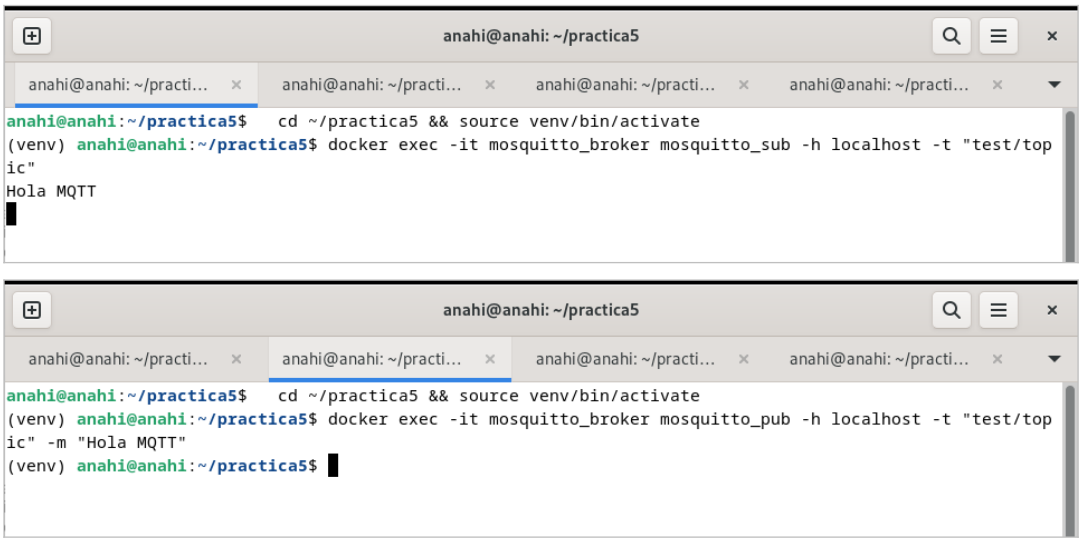


Figura 2. Prueba "Hola MQTT": mensaje recibido por el suscriptor (arriba) y comando del publicador (abajo).

Actividad 2 – Simulador de sensores IoT

- `simulador_sensores.py` simula tres vehículos (VH-001, VH-002, VH-003) con las funciones `simular_gps()` (lat, lng, speed), `simular_temperatura()` (−5 a 8 °C) y `simular_combustible()` (10 a 100 %).
- Cada mensaje es un JSON con `vehicle_id`, `timestamp` (ISO 8601) y la medición, y se publica con QoS 1 en el topic jerárquico `flota/{vehiculo}/{sensor}`.
- Una ronda publica 9 mensajes (3 vehículos × 3 sensores), con una pausa de 5 s por vehículo. El parámetro `--rondas N` limita la ejecución; sin él, el simulador corre hasta presionar Ctrl+C.

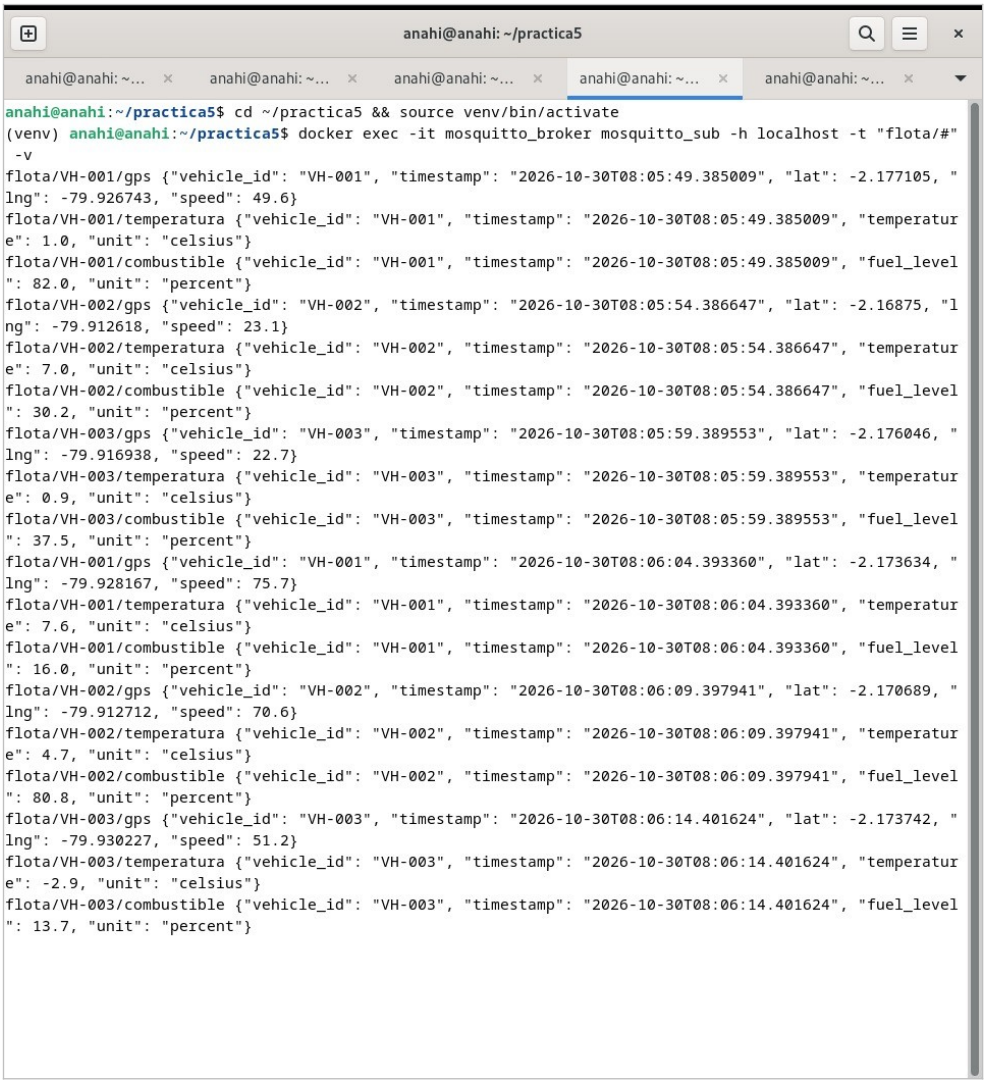


Figura 3. Mensajes recibidos en flota/# con los topics de los tres vehículos.

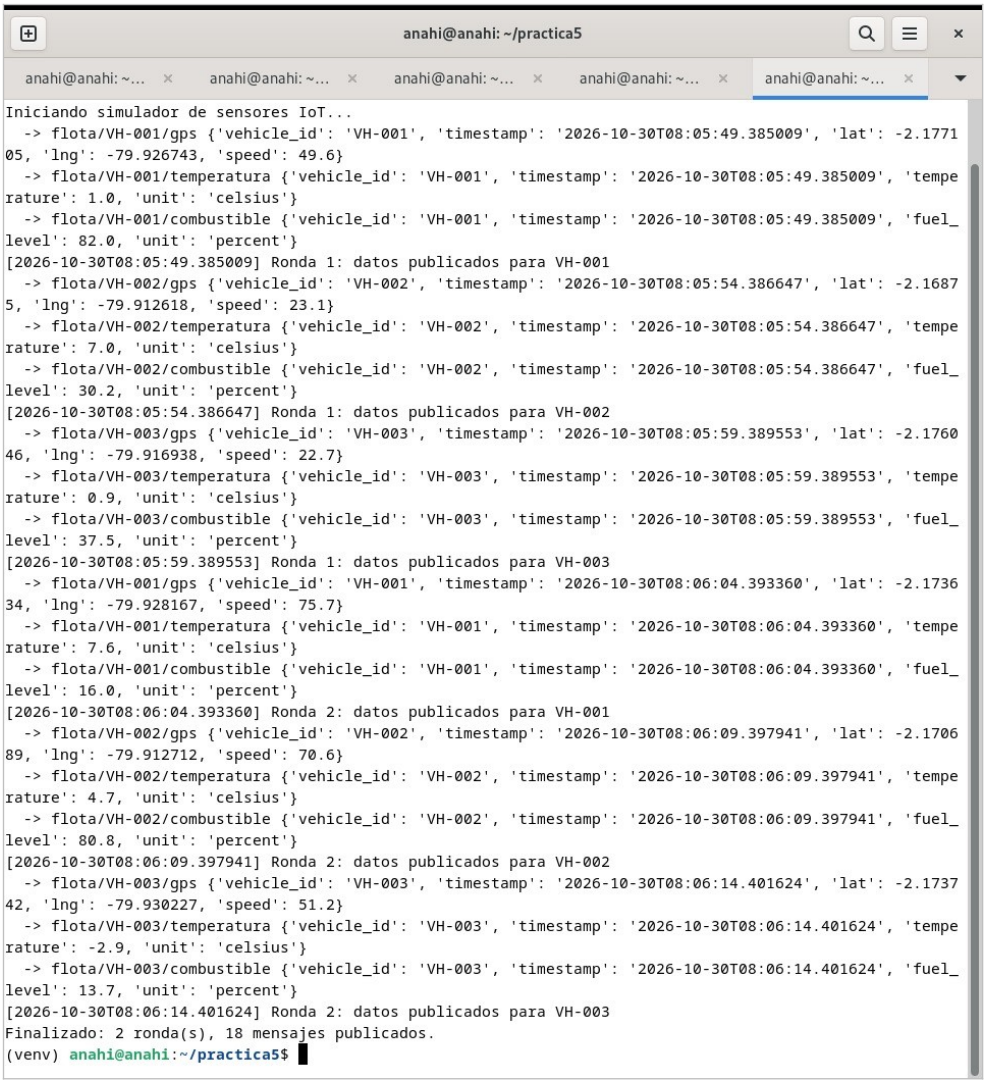


Figura 4. Ejecución del simulador: 2 rondas y 18 mensajes publicados.

Actividad 3 – Suscriptor MQTT con almacenamiento en SQLite

- `suscriptor_sqlite.py` se suscribe, dentro del evento `on_connect`, a `flota/+/gps`, `flota/+/temperatura` y `flota/+/combustible`. El comodín + representa a cualquier vehículo.
- Al recibir un mensaje: lee el topic, decodifica el JSON, identifica el sensor, inserta el registro en `telemetria.db` y evalúa si hay alerta.
- La base tiene tres tablas: `gps_data` (lat, lng, speed), `temp_data` (temperature, unit) y `fuel_data` (fuel_level, unit), todas con `id`, `vehicle_id` y `timestamp`.
- Reglas de alerta: temperatura **mayor que 4 °C** (cadena de frío) y combustible **menor que 20 %**. Se guardan tanto los datos normales como los que generan alerta.

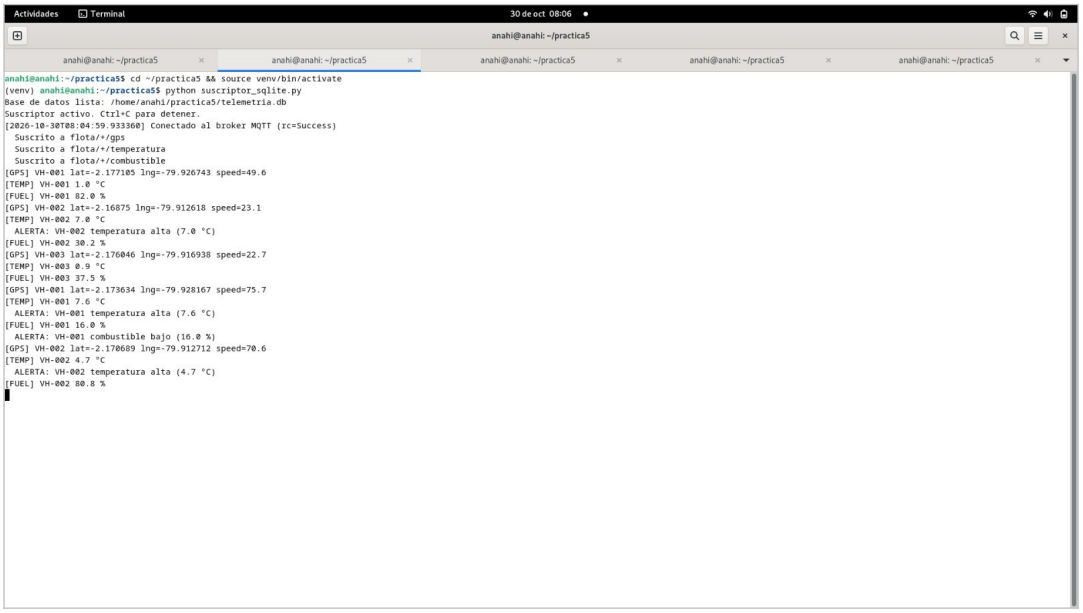


Figura 5. Suscriptor almacenando la telemetría y mostrando alertas de temperatura y combustible.

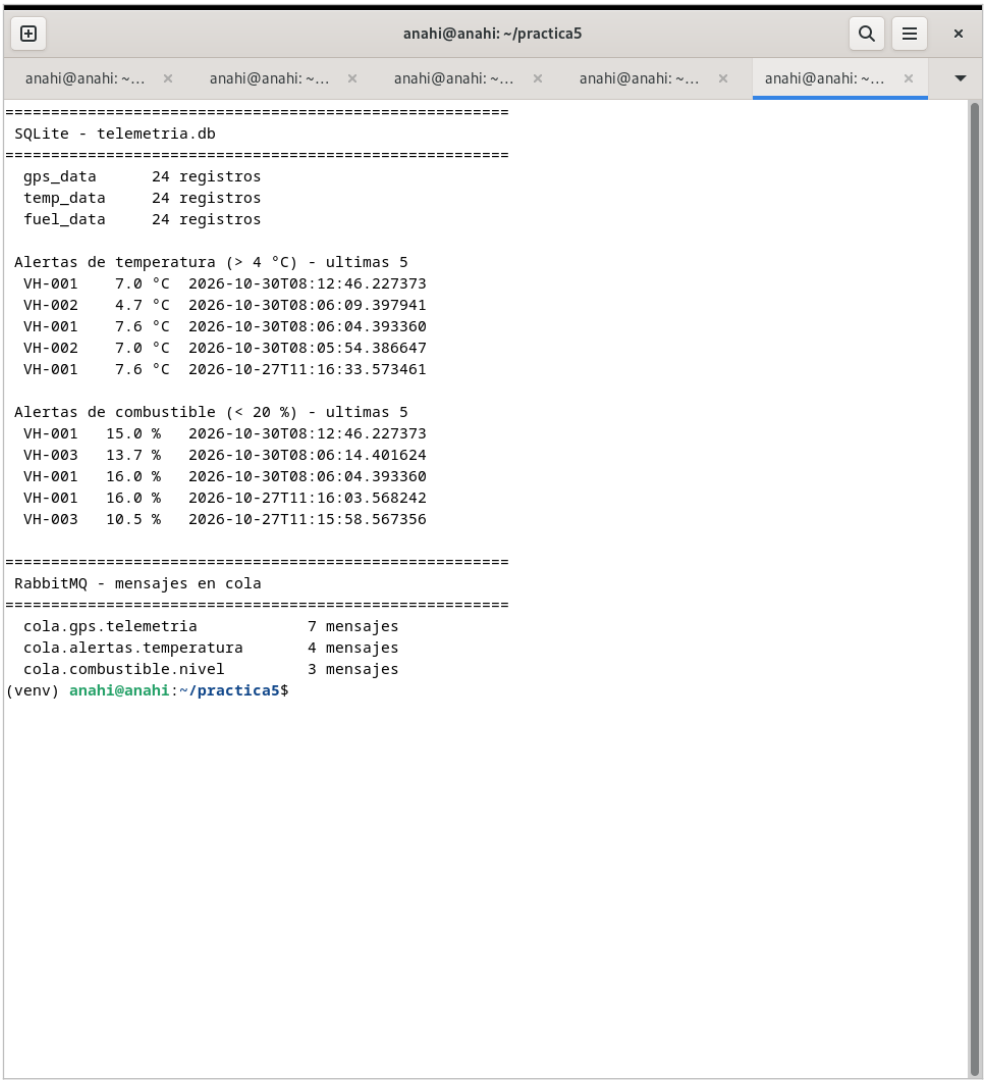


Figura 6. Verificación con verificar_resultados.py: registros en las tres tablas, alertas guardadas y mensajes en las colas.

Actividad 4 – Servicio puente MQTT → RabbitMQ

- `mqtt_rabbitmq_bridge.py` se conecta a RabbitMQ con pika y declara tres colas durables: `cola.gps.telemetria`, `cola.alertas.temperatura` y `cola.combustible.nivel`.
- El puente se suscribe a `flota/#` y publica mediante el exchange predeterminado (""), donde la routing key coincide con el nombre de la cola. Los mensajes son persistentes (`delivery_mode=2`).
- Reglas de enrutamiento: todo GPS va a `cola.gps.telemetria`. Si la temperatura es > 4 °C, se añade `type: TEMP_ALERT` y se envía a `cola.alertas.temperatura`. Si el combustible es < 20 %, se añade `type: FUEL_LOW` y se envía a `cola.combustible.nivel`.
- Prueba controlada con `prueba_alertas.py`: GPS con speed 42, temperatura 7 °C y combustible 15 %. Cada mensaje llegó a su cola correspondiente.
- Orden de ejecución: (1) `docker compose up -d`, (2) suscriptor, (3) puente, (4) simulador y prueba de alertas, (5) verificación.

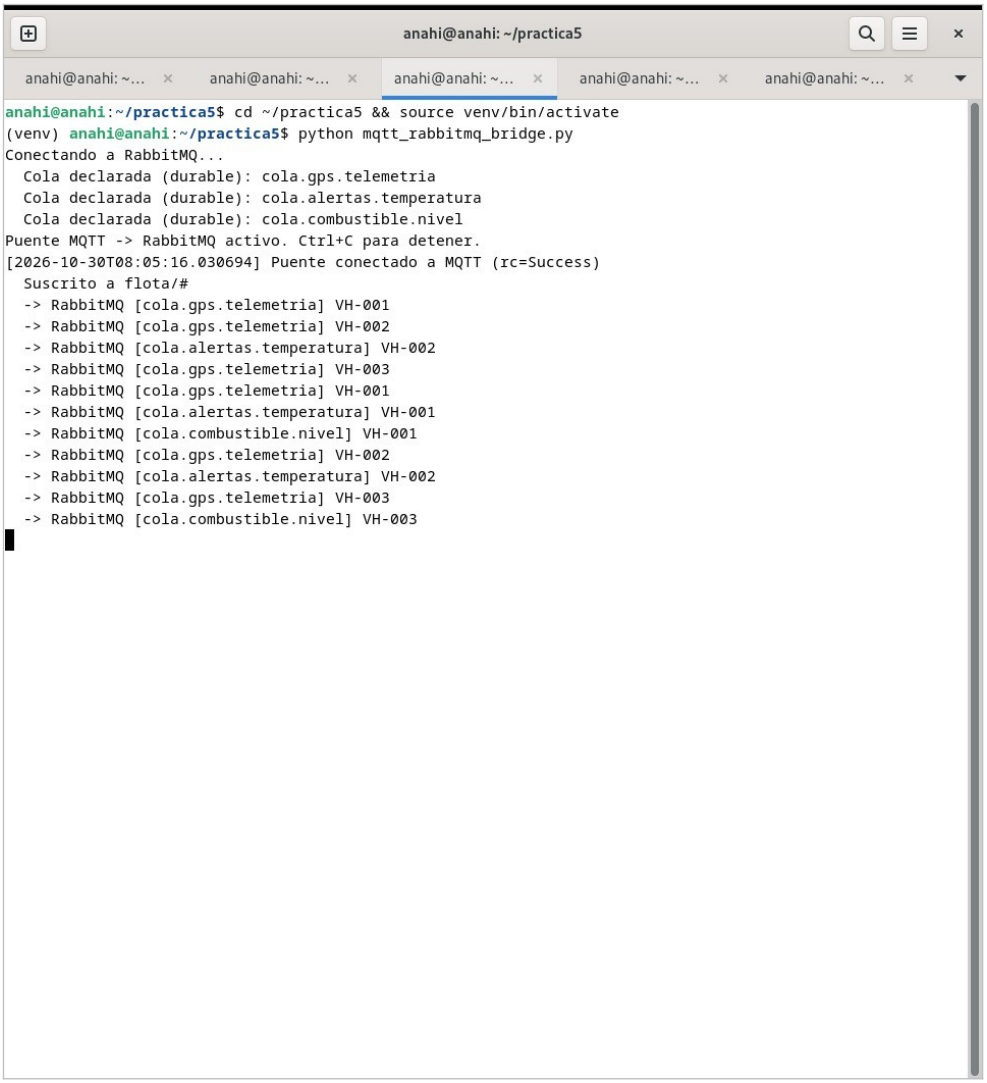


Figura 7. Puente MQTT → RabbitMQ: declaración de colas durables y reenvío según las reglas.

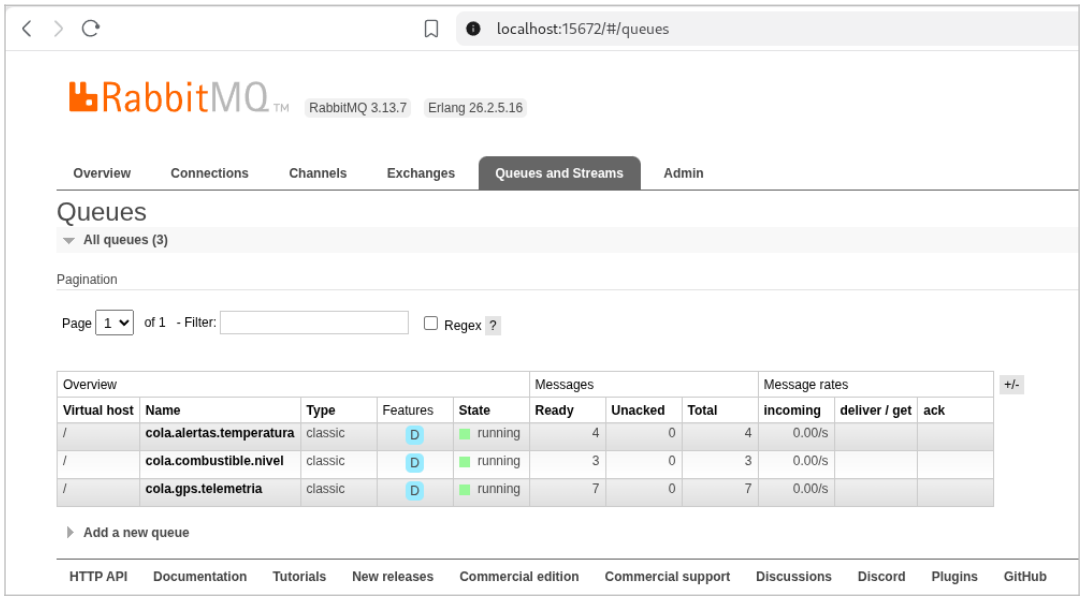


Figura 8. Panel de administración de RabbitMQ (Queues and Streams): tres colas durables (D) con 4, 3 y 7 mensajes.

5. Resultados:

- **Broker MQTT funcional:** Mosquitto corriendo en Docker, capaz de recibir y distribuir mensajes de múltiples publicadores a múltiples suscriptores (Figuras 1–3).
- **Simulador de sensores IoT:** script Python que simula la telemetría de tres vehículos (GPS, temperatura y combustible) y la publica periódicamente en topics MQTT jerárquicos, 9 mensajes por ronda (Figura 4).
- **Suscriptor con almacenamiento:** suscriptor Python que recibe la telemetría, detecta condiciones de alerta y la almacena en SQLite con una tabla por tipo de sensor. Al final de las pruebas, cada tabla tenía 24 registros (Figuras 5 y 6).
- **Servicio puente operativo:** servicio Python que consume datos de MQTT y los publica en colas de RabbitMQ, filtrando las alertas. Tras las pruebas, las colas tenían 7 mensajes de GPS, 4 alertas de temperatura y 3 de combustible (Figuras 6–8).

6. Conclusiones:

- Se implementó la capa IoT del sistema de monitoreo de flota cumpliendo el objetivo de la práctica: Mosquitto en Docker recibe la telemetría simulada, un suscriptor la persiste en SQLite con alertas por umbral y un puente reenvía a RabbitMQ solo los datos que el backend debe procesar.
- La jerarquía `flota/{vehiculo}/{sensor}` y los comodines `+` y `#` desacoplan a publicadores y suscriptores. Un mismo broker sirvió a dos consumidores independientes (suscriptor y puente) sin modificar el simulador.
- MQTT y AMQP se complementan: MQTT es ligero y adecuado para dispositivos IoT, mientras que RabbitMQ aporta colas durables, mensajes persistentes y enrutamiento hacia el backend, como se comprobó con las tres colas.
- El orden de arranque importa: con la configuración usada, los mensajes publicados antes de que exista una suscripción se pierden. Por eso el suscriptor y el puente deben iniciarse antes que el simulador.
- Desplegar con Docker exige verificar los recursos del host: los conflictos de puertos y la falta de espacio en disco impidieron arrancar los servicios hasta resolverlos.

7. Recomendaciones

- Desactivar `allow_anonymous` y configurar usuarios y contraseñas (y TLS en el puerto 8883) en Mosquitto, y cambiar las credenciales `guest/guest` de RabbitMQ antes de usar el sistema fuera del laboratorio.
- Usar QoS 1 o 2 y sesiones persistentes (`clean_session=False`) en el suscriptor y el puente, para no perder mensajes publicados mientras estén desconectados.

- Definir los umbrales de alerta en un archivo de configuración o en variables de entorno, en lugar de dejarlos fijos en el código, para ajustarlos por tipo de carga.
- Contenerizar también los scripts de Python y agregar `healthcheck` en Docker Compose, para que el puente espere a que RabbitMQ esté listo.
- Revisar el espacio en disco (`df -h`, `docker system df`) antes de la práctica, ya que RabbitMQ se detiene o falla al arrancar si el disco está casi lleno.

8. Preguntas de Control

- **¿Explique las diferencias clave entre MQTT y AMQP en términos de protocolo, patrones de comunicación y casos de uso típicos. ¿Por qué MQTT es más adecuado para IoT?**

  Protocolo: MQTT es un protocolo binario muy ligero sobre TCP, con una cabecera fija de 2 bytes y pocos tipos de paquete. AMQP 0-9-1 es más complejo: maneja conexiones con canales, frames, propiedades de mensaje y confirmaciones (ack), y está pensado para la integración empresarial.

  Patrones de comunicación: MQTT usa solo publicador/suscriptor por topics; el broker reenvía cada mensaje a todos los suscriptores interesados. AMQP usa el modelo exchange → binding → cola, que permite pub/sub, colas de trabajo con competing consumers, enrutamiento por reglas (direct, topic, fanout, headers) y RPC.

  Casos de uso: MQTT se usa en sensores, telemetría, domótica y dispositivos móviles. AMQP/RabbitMQ se usa en microservicios, procesamiento de pedidos, tareas en segundo plano y distribución de alertas en el backend.

  ¿Por qué MQTT para IoT? Consume poco ancho de banda, memoria y batería, y tolera redes inestables gracias a los niveles de QoS, las sesiones persistentes, los mensajes retenidos y el Last Will. Además, el sensor solo publica en un topic sin conocer a los consumidores. En esta práctica, MQTT recoge la telemetría de los vehículos y RabbitMQ lleva al backend solo lo que debe procesarse.

- **¿Por qué MQTT organiza los mensajes en topics jerárquicos, mientras que RabbitMQ usa exchanges, colas y bindings? ¿Qué ventajas ofrece cada enfoque?**

  MQTT (topics jerárquicos): el topic actúa como una dirección con niveles separados por "/", por ejemplo `flota/VH-001/temperatura`. El broker no almacena colas por consumidor, sino que filtra por topic, y los comodines simplifican la suscripción: `flota/+/temperatura` recibe la temperatura de todos los vehículos y `flota/#` recibe toda la flota. Ventajas: es simple, no requiere configuración previa en el broker (los topics se crean al publicar) y escala bien con muchos dispositivos.

  RabbitMQ (exchanges, colas y bindings): separa al productor del destino. El productor publica en un exchange; los bindings y la routing key deciden en qué colas se deposita el mensaje, y la cola lo **almacena** hasta que un consumidor lo procesa y confirma. Ventajas: persistencia (colas durables y mensajes con `delivery_mode=2`), entrega garantizada con ack, balanceo

de carga entre varios consumidores de la misma cola (competing consumers) y enrutamiento flexible.

En la práctica se combinaron ambos enfoques: el puente lee `flota/#` en MQTT y, usando el exchange predeterminado (routing key = nombre de la cola), envía todo el GPS a `cola.gps.telemetria` y solo las alertas a `cola.alertas.temperatura` y `cola.combustible.nivel`.

- **¿Qué niveles de QoS ofrece MQTT? Explique las ventajas y desventajas de cada uno y dé un ejemplo de uso en el sistema de flota.**

  QoS 0 – como máximo una vez: se envía sin confirmación. Ventaja: es el más rápido y con menor consumo. Desventaja: el mensaje puede perderse. Ejemplo: las posiciones GPS frecuentes, porque si se pierde una, la siguiente llega en pocos segundos.

  QoS 1 – al menos una vez: el receptor confirma con PUBACK y el emisor reintenta si no recibe la confirmación. Ventaja: garantiza la entrega con poca sobrecarga. Desventaja: puede haber duplicados, que el consumidor debe tolerar. Ejemplo: el nivel de combustible y la telemetría general; es el nivel que usa el simulador de esta práctica.

  QoS 2 – exactamente una vez: usa un intercambio de cuatro pasos (PUBLISH, PUBREC, PUBREL, PUBCOMP). Ventaja: no hay pérdidas ni duplicados. Desventaja: es el más lento y genera más tráfico. Ejemplo: las alertas críticas de cadena de frío (temperatura > 4 °C), donde perder o duplicar una alerta tiene consecuencias reales sobre la carga.

9. Declaración de uso de IA

Para el desarrollo de la presente práctica se emplearon herramientas de Inteligencia Artificial con fines de apoyo académico y técnico. Se utilizó Claude (Anthropic) como asistente para interpretar la guía de actividades, resolver incidencias del entorno (puerto 1883 ocupado, espacio en disco y error de cookie de RabbitMQ), elaborar y depurar los scripts en Python y apoyar la redacción del informe. El código fue ejecutado y verificado por el equipo en su propio equipo, como muestran las capturas.

Prompt:

"Analiza todo este documento [Guía de Actividades Nro. 5 de Sistemas Distribuidos] y dime qué se debe hacer paso a paso."

10. Bibliografía

[1] OASIS, "MQTT Version 5.0," OASIS Standard, mar. 2019. [En línea]. Disponible: https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html

[2] RabbitMQ, "AMQP 0-9-1 Model Explained," RabbitMQ Documentation. [En línea]. Disponible: https://www.rabbitmq.com/tutorials/amqp-concepts

[3] Eclipse Foundation, "paho-mqtt: MQTT version 5.0/3.1.1 client class," Python Package Index. [En línea]. Disponible: https://pypi.org/project/paho-mqtt/

[4] Pika, “Introduction to Pika,” Pika Documentation. [En línea]. Disponible: https://pika.readthedocs.io/

[5] Eclipse Foundation, “mosquitto.conf – the configuration file for mosquitto,” Eclipse Mosquitto. [En línea]. Disponible: https://mosquitto.org/man/mosquitto-conf-5.html

[6] M. van Steen y A. S. Tanenbaum, Distributed Systems, 4.ª ed. distributed-systems.net, 2023.